

Einführung

Vertex-Buffer-Object (VBO)

Ein VBO speichert Vertex-Daten (wie Positionen, Farben, Texturkoordinaten, Normalen, ...) im Grafikspeicher der GPU. Es ermöglicht eine schnelle und wiederholte Nutzung von Vertex-Daten und reduziert damit die Datenübertragung zwischen CPU und GPU. Die Daten liegen hintereinander im Speicher, wobei die Struktur der Daten durch die Vertex-Attribut-Definitionen in einem Vertex-Array-Objekt (VAO) festgelegt wird.

Vertex-Array-Object (VAO)

Ein VAO speichert den Zustand von Vertex-Attributen und deren Zuordnung zu Vertex-Puffern. Es definiert, wie die Vertex-Daten interpretiert werden sollen, indem es die Verknüpfung zwischen Vertex-Attributen und den entsprechenden Vertex-Buffer-Objekten (VBOs) herstellt. Ein VAO ermöglicht es, die Konfiguration von Vertex-Attributen zu speichern und wiederzuverwenden, ohne sie jedes Mal neu konfigurieren zu müssen.

```
cmake --build build/  
./build/Debug/task02.exe
```

Aufgabe02

<https://docs.gl/>

glCreateBuffers(...)

```
void glCreateBuffers(  
    GLsizei n,  
    GLuint *buffers  
);
```

- n: Anzahl der zu erzeugenden Buffer-Objekte.
- buffers: Zeiger auf ein Array, in dem die Namen der neuen Buffer-Objekte gespeichert werden.

Wird verwendet, um ein oder mehrere Buffer-Objekte als Speicherbereiche auf der GPU zu erzeugen. Jedes Buffer-Objekt besitzt dabei eine eindeutige ID. Die Funktion ist Teil des Direct State Access (DSA)-Ansatzes, der es ermöglicht, Objekte unabhängig von der aktuellen Bindung zu erstellen und zu konfigurieren.

Der Unterschied zu glGenBuffers(...) liegt darin, dass ein Objekt zuerst gebunden werden muss, bevor es konfiguriert oder verwendet werden kann. Die Funktion gehört zur älteren OpenGL-API (vor DSA) und erzeugt nur Namen für Buffer-Objekte, ohne sie zu initialisieren. Für die Nutzung muss das Buffer-Objekt erst mit glBindBuffer an einen Zieltyp (bspw. GL_ARRAY_BUFFER) gebunden werden.

glNamedBufferSubData(...)

```
void glNamedBufferSubData(  
    GLuint buffer,  
    GLintptr offset,  
    GLsizeiptr size,  
    const void *data  
);
```

- buffer: Name/ ID vom Buffer-Objekt.

- offset: Gibt den Offset in die Daten des Buffer-Objekts in Bytes an, wo die Aktualisierung beginnen soll.
- size: Größe in Bytes des Bereichs, welcher aktualisiert werden soll.
- data: Pointer zu den neuen Daten, die in das Buffer-Objekt kopiert werden sollen.

Wird verwendet, um einen Teilbereich eines bereits existierenden Buffer-Objekts zu aktualisieren, ohne den gesamten Puffer neu zu laden. Die Funktion benötigt keine vorherige Bindung des Buffer-Objekts, wie es bei `glBufferSubData` der Fall wäre. Stattdessen wird das Zielobjekt direkt über seine ID angegeben.

glCreateVertexArrays(...)

```
void glCreateVertexArrays(  
    GLsizei n,  
    GLuint *arrays  
);
```

- n: Anzahl der zu erstellenden Vertex-Array-Objekte.
- arrays: Ein Zeiger auf ein Array, in dem die generierten VAO-Namen (IDs) gespeichert werden.

Wird verwendet, um Vertex-Array-Objekte (VAO) zu erstellen. Erstellt eine oder mehrere VAOs und speichert die Namen (IDs) der VAOs im übergebenen Array. VAOs speichern den Zustand von Vertex-Attributen und deren Zuordnung zu Vertex-Puffern. Die Funktion ist ebenso Teil des Direct State Access (DSA)-Ansatzes und die erzeugten VAOs müssen nicht gebunden werden, um sie zu konfigurieren.

glVertexArrayVertexBuffer(...)

```
void glVertexArrayVertexBuffer(  
    GLuint vaobj,  
    GLuint bindingindex,  
    GLuint buffer,  
    GLintptr offset,  
    GLsizei stride  
);
```

- vaobj: Name/ ID des zu konfigurierenden Vertex-Array-Objekts (VAO).
- bindingindex: Binding-Index mit dem die Vertex-Attribute mit dem Vertex-Buffer verbunden werden.
- buffer: ID des Vertex-Buffer-Objekts (VBO), der an den Binding-Index gebunden werden soll.
- offset: Byte-Offset im Buffer, ab dem die Daten gelesen werden sollen.
- stride: Größe in Bytes eines einzelnen Vertex-Datensatzes im Buffer. Gibt an, wie weit die Daten für aufeinanderfolgende Vertices im Puffer voneinander entfernt sind.

Wird verwendet, um ein Vertex-Buffer-Objekt (VBO) mit einem Vertex-Array-Objekt (VAO) zu verknüpfen. Legt fest, welcher Vertex-Buffer für einen bestimmten Binding-Index eines VAOs verwendet werden soll. Die Funktion ist Teil der DSA-API und ermöglicht eine effiziente Verwaltung von Vertex-Daten ohne vorheriges Binding.

glVertexArrayAttribBinding(...)

```
void glVertexArrayAttribBinding(  
    GLuint vaobj,  
    GLuint attribindex,
```

```
    GLuint bindingindex  
);
```

- vaobj: Name/ ID des zu konfigurierenden Vertex-Array-Objekts (VAO).
- attribindex: Index des Vertex-Attributs, das mit einem Binding-Index verknüpft werden soll.
- bindingindex: Binding-Index, der mit dem Attribut verknüpft wird.

Wird verwendet, um ein Vertex-Attribut mit einem Binding-Index eines Vertex-Array-Objekts (VAO) zu verknüpfen. Legt fest, welcher Binding-Index für ein bestimmtes Vertex-Attribut verwendet werden soll. Der Binding-Index gibt an, welcher Vertex-Puffer (VBO) die Daten für das Attribut bereitstellt.

glVertexArrayAttribFormat(...)

```
void glVertexArrayAttribFormat(  
    GLuint vaobj,  
    GLuint attribindex,  
    GLint size,  
    GLenum type,  
    GLboolean normalized,  
    GLuint relativeoffset  
);
```

- vaobj: Name/ ID des zu konfigurierenden Vertex-Array-Objekts (VAO).
- attribindex: Index des Vertex-Attributs, dessen Format festgelegt werden soll.
- size: Anzahl der Komponenten pro Vertex-Attribut (1 bis 4).
- type: Datentyp der Komponenten (z.B. GL_INT, GL_FLOAT, GL_UNSIGNED_BYTE).
- normalized: Gibt an, ob für Ganzzahlen (GL_INT, ...) in den Bereichen [0, 1] (für unsigned) oder [-1, 1] (für signed) normalisiert werden sollen.
- relativeoffset: Byte-Offset des Attributs innerhalb eines Vertex-Datensatzes.

Wird verwendet, um das Datenformat eines Vertex-Attributs zu definieren. Legt fest, wie die Daten für ein bestimmtes Vertex-Attribut interpretiert werden sollen. Bestandteile der Definition sind: die Anzahl der Komponenten, der Datentyp, die Normalisierung und der Offset innerhalb eines Vertex-Datensatzes. Ein Vertex-Buffer kann Vertex-Attribute mit unterschiedlichen Datentypen beinhalten.

glEnableVertexArrayAttrib(...)

```
void glEnableVertexArrayAttrib(  
    GLuint vaobj,  
    GLuint index  
);
```

- vaobj: Name/ ID des zu konfigurierenden Vertex-Array-Objekts (VAO).
- index: Index des Vertex-Attributs, das aktiviert werden soll.

Wird verwendet, um ein Vertex-Attribut in einem Vertex-Array-Objekt (VAO) zu aktivieren. Durch die Aktivierung des Vertex-Attributs eines VAOs wird dieses in der OpenGL-Pipeline verwendet. Im deaktivierten Zustand wird das Attribut von der Pipeline ignoriert.

glBindVertexArray(...)

```
void glBindVertexArray(  
    GLuint array  
);
```

- array: Name/ ID des zu bindenden Vertex-Array-Objekts (VAO). Wenn array den Wert 0 hat, wird das aktuelle VAO entbunden.

Wird verwendet, um ein Vertex-Array-Objekt (VAO) an den aktuellen OpenGL-Kontext zu binden. Gehört zur älteren OpenGL-API (vor DSA). Durch das Binden wird das VAO aktiviert, sodass alle nachfolgenden Befehle, die Vertex-Attribute oder Vertex-Buffer betreffen, auf dieses VAO angewendet werden.

glDrawArrays(...)

```
void glDrawArrays(  
    GLenum mode,  
    GLint first,  
    GLsizei count  
);
```

- mode: Gibt den Typ der zu zeichnenden Primitiven an (z.B. GL_POINTS, GL_LINES, ...).
- first: Index des ersten Vertex-Datensatzes im Buffer, der verwendet werden soll.
- count: Anzahl der zu verwendenden Vertices (ab dem Index first).

Wird verwendet, um Primitiven (wie Punkte, Linien oder Dreiecke) aus den aktuell gebundenen Vertex-Daten zu rendern. Die Vertex-Daten kommen aus den aktuell gebundenen Vertex-Array-Objekten (VAOs) und Vertex-Buffer-Objekten (VBOs). Die Funktion verwendet die Vertex-Daten in der Reihenfolge, in der sie im Buffer gespeichert sind.

Ergebnis

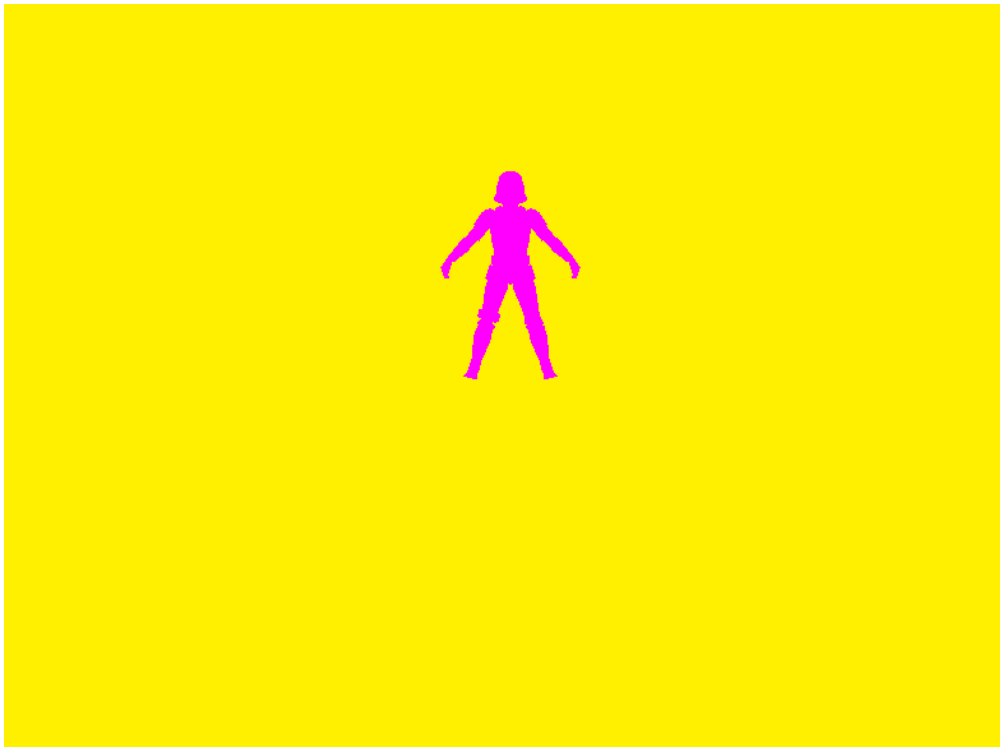


Figure 1: Ergebnis von Aufgabe 02